

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture -4



By- Vishvadeep Gothi sir

Recap of Previous Lecture

Doubt ?



Topic

Peterson's Solution

Topic

Hardware Solutions of Synchronization

Topic

Test-And-Set()

Topic

Swap()

Topics to be Covered



Topic

Semaphore

Question 1

```
turn=0;
```

```
while(true)
{
    while(turn);
    turn=1;
    //CS
    turn=0;
    RS;
}
```

X M.E.

✓ Progress

X B.W.

```
while(true)
{
```

```
    while(turn);
```

```
    turnlock=1;
```

```
    //CS
```

```
    turnlock=0;
```

```
    RS;
```

```
}
```

Question 2

```
lock=False;
```

```
while(true)
{
    while(lock!=False);
    CS
    lock=True;
    RS;
}
```

strict alternation

✓ m.c.
 ✗ Progress
 ✓ B.W.

```
while(true)
{
    while(lock!=True);
    CS
    lock=False;
    RS;
}
```



Topic : Synchronization Tool

✓ 1. Semaphore

→ platform & h/w independent

2. Monitor

→ used by programmers to provide their own synch.





Topic : Synchronization Tool

Semaphore

- Integer value which can be accessed using following functions only
 1. `wait()` / `P()` / `Degrade()`
 2. `signal()` / `V()` / `Upgrade()`

} $\Rightarrow$ atomic functions





Topic : wait() & signal()



wait(S)

```
{  
    while(S<=0);  
    S--;  
}
```



it decrements semaphore
by 1 if semaphore is
1 or greater than 1.

signal(S)

```
{  
    S++;  
}
```





Topic : Types of Semaphore

Binary Semaphore	Counting Semaphore
Can have 2 values ↓ 0, 1	Can have any +ve int value. ↓ 0, 1, 2, 3, 4, 5, 6, ...

⇒ By default semaphores are non-negative int value.



Topic : Types of Semaphore

Binary Semaphore	Counting Semaphore
It is used to implement the solution of critical section problems with multiple processes	It is used to control access to a resource that has multiple instances

↓
mutual exclusion



Topic : wait() & signal() on Binary Semaphores

if $S = 0$

wait(S) $\Rightarrow$ Can not complete successfully

if $S = 1$

wait(S) $\Rightarrow$ successfully completed and S becomes zero.

if $S = 0$

signal(S) $\Rightarrow$ successfully completed and S becomes 1.

if $S = 1$

signal(S) $\Rightarrow$ successfully completed and S remains 1.



Topic : Critical Section Solution

Binary semaphore

$S = 1$

P_0, P_1, P_2

```
while(True)
```

```
{
```

```
    wait(S)
```

```
    C.S.
```

```
    signal(s)
```

```
}
```

[NAT]



#Q. Consider a counting semaphore S, initialized with value 10. What should be the value of S after executing 6 times P() and 8 time V() function on S?

$$10 - 6 + 8 = \underline{\underline{12}} \text{ Ans.}$$

#Q. Consider a semaphore S, initialized with value 37. Which of the following options gives the final value of S=12?

A Execution of 22 P() and 15 V() $37 - 22 + 15 = 30$

B ✓ Execution of 25 P() $37 - 25 = 12$

C ✓ Execution of 33 P() and 8 V() $37 - 33 + 8 = 12$

D ✓ Execution of 31 P() and 6 V() $37 - 31 + 6 = 12$

Binary semaphore $S = 1$

ex:-

P1
wait(s)
c.s.
signal(s)

P2
signal(s)
c.s.
wait(s)

No mutual exclusion

#Q. Consider a binary semaphore S, initialized with value 1. Consider 10 processes P1, P2 P10. All processes have same code as given below but, one process P10 has signal(S) in place of wait(S). If all processes to be executed only once, then maximum number of processes which can be in critical section together ?

```
process
{
    wait(S)
    C.S.
    signal(s)
}
```

Ans = 3



Topic : Solution



✓ P1, P2,, P9

```
process
{
    wait(S)
    C.S.
    signal(s)
}
```

✓ P10

```
process
{
    signal(S)
    C.S.
    signal(s)
}
```

$S = 1$

$P1 \Rightarrow \text{wait}(S) \Rightarrow S=0$
into C.S.

$P10 \Rightarrow \text{signal}(S) \Rightarrow S=1$
into C.S.

$P2 \Rightarrow \text{wait}(S) \Rightarrow S=0$
into C.S.

[MCQ]



#Q. Consider a binary semaphore S, initialized with value 1. Consider 10 processes P1, P2 P10. All processes have same code as given below but, one process P10 has signal(S) in place of wait(S). If all processes to be executed only once, then maximum number of processes which can be in critical section together ?

```
while(True)
{
    wait(S)
    C.S.
    signal(s)
}
```

10



Topic : Solution



P1, P2,, P9

```
while(True)
{
    wait(S)
    C.S.
    signal(s)
}
```

P10

```
while(True)
{
    signal(S)
    C.S.
    signal(s)
}
```

$P_1 \Rightarrow \text{wait}(s) \Rightarrow s=0 \Rightarrow \text{in CS}$

$P_{10} \Rightarrow \text{signal}(s) \Rightarrow s=1$

$P_2 \Rightarrow \text{wait}(s) \Rightarrow s=0 \text{ in CS}$

$P_{10} \Rightarrow \text{signal}(s) \Rightarrow s=1$

$P_3 \Rightarrow \text{wait}(s) \Rightarrow s=0 \text{ in CS}$

$P_{10} \Rightarrow \text{signal}(s) \Rightarrow s=1$

$P_4 \Rightarrow \text{wait}(s) \Rightarrow s=0 \text{ in CS}$

;

[Question]



#Q. Consider a scenario where 2 processes are available: P0 and P1. The processes must be executed in order P0 then P1 then P0 then P1 and this sequence continues forever. Write a piece of code to control the sequence of execution with minimum number of ^{binary}semaphores?

$S_0 = 1, S_1 = 0$

P₀
wait(S_0)
=====
signal(S_1)

P₁
wait(S_1)
=====
signal(S_0)

[Question]



#Q. Consider a scenario where 3 processes are available: P0, P1 and P2. The processes must be executed in order P1, P0 and P2 only. Write a piece of code to control the sequence of execution with minimum number of semaphores?

$P_1 \rightarrow P_0 \rightarrow P_2 \rightarrow P_1 \rightarrow P_0 \rightarrow P_2 \dots$

$S_0 = 0$
 $S_1 = 1$
 $S_2 = 0$

P₀
wait(S_0)
=====
signal(S_2)

P₁
wait(S_1)
=====
signal(S_0)

P₂
wait(S_2)
=====
signal(S_1)

[MCQ]



#Q. Consider the following threads, T_1 , T_2 , and T_3 executing on a single processor, synchronized using three binary semaphore variables, S_1 , S_2 , and S_3 , operated upon using standard `wait()` and `signal()`. The threads can be context switched in any order and at any time. Which initialization of the semaphores would print the sequence BCABCABCA...?

[2022]

T1	T2	T3
<pre>while(true) { wait (S₃); print("C"); signal(S₂); }</pre>	<pre>while(true){ wait(S₁); print("B"); signal(S₃);} </pre>	<pre>while(true) { wait(S₂); print("A"); signal(S₁); }</pre>

$T_2 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_1 \rightarrow T_3$

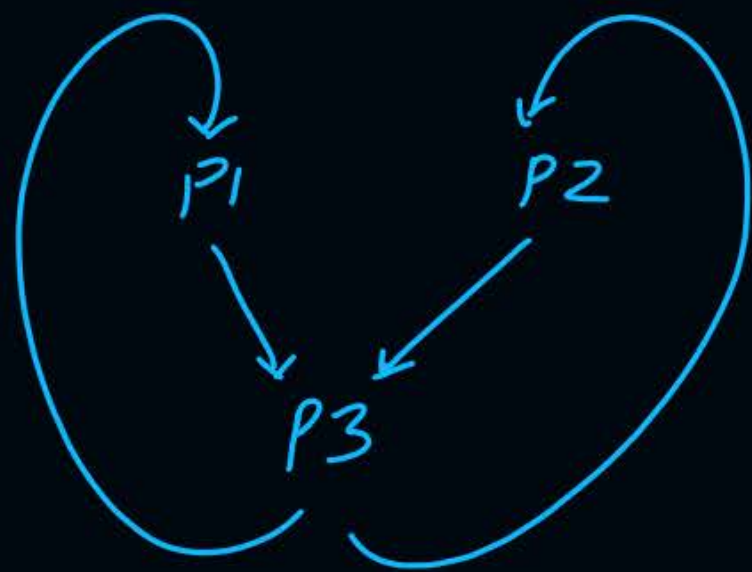
A $S_1 = 1; S_2 = 1; S_3 = 1$

B $S_1 = 1; S_2 = 1; S_3 = 0$

C ✓ $S_1 = 1; S_2 = 0; S_3 = 0$

D $S_1 = 0; S_2 = 1; S_3 = 1$

Ques) $\rightarrow$ processes P_1, P_2, P_3
 P_3 can execute only when P_1 and P_2 completes.
 P_1, P_2 can execute again only when P_3 completes.

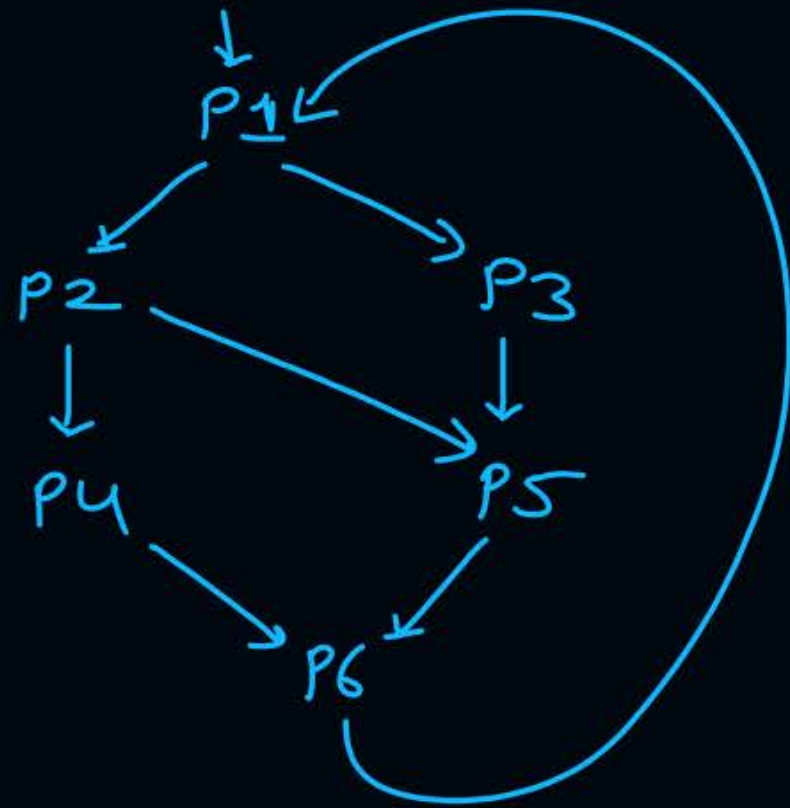


$S_1 = 1$

$S_2 = 1$

P_1	P_2	P_3
wait(S_1)	wait(S_2)	wait(S_{31})
$\equiv$	$\equiv$	wait(S_{32})
$\equiv$	$\equiv$	$\equiv$
signal(S_{31})	signal(S_{32})	$\equiv$
		signal(S_1)
		signal(S_2)

Ques)



$S1 = 1$
 $S2 = 0$
 $S3 = 0$
 $S4 = 0$
 $S51 = 0$
 $S52 = 0$
 $S61 = 0$
 $S62 = 0$

<u>P1</u>	<u>P2</u>	<u>P3</u>	<u>P4</u>
wait(S1)	wait(S2)	wait(S3)	wait(S4)
≡	≡	≡	≡
signal(S2)	signal(S4)	signal(S52)	signal(S61)
signal(S3)	signal(S51)		

<u>P5</u>	<u>P6</u>
wait(S51)	wait(S61)
wait(S52)	wait(S62)
≡	≡
signal(S62)	signal(S1)



2 mins Summary



Topic

Semaphore





Happy Learning

THANK - YOU

